INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO
CAMPUS CATANDUVA — CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

TILA HEALTH TRIAGE: CLASSIFICAÇÃO DE RISCO EM PRONTO ATENDIMENTO

Desenvolvimento e Avaliação de Modelos de IA com Deploy no Streamlit Cloud

Disciplina:	Tópicos Avançados em Inteligência Artificial
Professor:	Prof. Dr. Márcio Andrey Teixeira
Integrantes / Alunos:	Ryan Aguiar e Pedro Henrique Oliveira Pereira
Repositório no GitHub:	https://github.com/RyanAguiiarr/tila-health-triage
Aplicação Publicada:	https://tila-health-triage-7dupev3q3cmsbqxsks8t9x.streamlit.app/
Data de Entrega:	26 de outubro de 2026

1. Problema e Objetivo

Quem já precisou de atendimento em uma Unidade de Pronto Atendimento (UPA) ou hospital público sabe como a triagem inicial é um momento crítico. Quando chegam muitos pacientes ao mesmo tempo, a equipe de enfermagem precisa tomar decisões rápidas para definir quem entra imediatamente para a sala de emergência e quem pode aguardar na recepção sem risco de piora. Se essa classificação falhar, um paciente com sinais de infarto, sepse ou hipóxia grave pode esperar demais e ter complicações severas.

Pensando nisso, desenvolvi o **Tila Health Triage** como projeto prático da nossa disciplina. O objetivo é criar uma aplicação funcional de Inteligência Artificial que auxilie o profissional de saúde na recepção. O sistema recebe 9 dados simples aferidos no acolhimento (idade, pressão, frequência cardíaca, saturação de oxigênio, temperatura, dor, histórico e forma de chegada) e sugere a prioridade do atendimento em três níveis clínicos: Baixa (Verde), Média (Amarelo) e Alta Prioridade (Vermelho).

A proposta não é substituir a decisão do enfermeiro ou do médico, mas sim funcionar como uma segunda opinião ágil e baseada em dados, reduzindo dúvidas na triagem e ajudando a organizar a fila de espera de forma mais justa e segura.

2. Dataset: Origem, Atributos e Limitações

Para treinar os modelos em um cenário realista, utilizei o dataset público *Synthetic Medical Triage Priority Dataset*, disponibilizado na plataforma Kaggle por Emirhan Akku■. A base possui **18.000 registros** de atendimentos de emergência, contemplando sinais vitais universais que qualquer posto ou hospital afere na triagem.

Atributo	Tipo	Unidade	Faixa de Valores	O que Representa no Atendimento
age	Numérico	Anos	0.0 a 95.0	Idade do paciente no momento da consulta.
heart_rate	Numérico	BPM	33.4 a 152.3	Frequência cardíaca (batimentos por minuto).
systolic_blood_pressure	Numérico	mmHg	65.8 a 219.7	Pressão arterial sistólica (máxima).
oxygen_saturation	Numérico	%	79.5 a 100.0	Saturação de oxigênio no sangue (SpO■).
body_temperature	Numérico	°C	34.5 a 41.1	Temperatura corporal axilar.
pain_level	Numérico	Escala 1-10	1 a 10	Nível de dor relatado pelo próprio paciente.
chronic_disease_count	Numérico	Quantidade	0 a 10	Número de doenças crônicas prévias.
previous_er_visits	Numérico	Quantidade	0 a 11	Idas anteriores ao pronto-socorro.

arrival_mode	Categórico	Código	0, 1 ou 2	Modo de chegada: 0=a pé/carro, 1=cadeira, 2=ambulância.
prioridade (Alvo)	Target	Classe	0, 1 ou 2	0: Baixa (Verde); 1: Média (Amarelo); 2: Alta (Vermelho).

Distribuição dos Dados e Limpeza: Ao verificar a base com dfBruto.isna().mean(), constatei que não havia valores nulos em nenhuma das 10 colunas originais. Na variável de chegada (arrival_mode), fiz o mapeamento textual para números (0 para quem veio por conta própria, 1 para cadeira de rodas e 2 para ambulância). No alvo original (triage_level de 0 a 3), agrupei para os 3 níveis clássicos do Ministério da Saúde: **Classe 0 (Baixa/Verde)** com 9.924 casos (55,13%), **Classe 1 (Média/Amarelo)** com 7.185 casos (39,92%) e **Classe 2 (Alta/Vermelho)** com 891 atendimentos (4,95%).

Limitações Identificadas: Uma limitação importante é que os dados capturam apenas o momento da entrada. Sintomas que evoluem rápido ou alergias medicamentosas graves não estão descritos em números, o que reforça que a IA não pode atuar de forma 100% autônoma.

3. Metodologia: Preparação e Treinamento dos Modelos

A construção do código seguiu o passo a passo que aprendemos nas aulas do Prof. Márcio, estruturado em um script Python claro e reprodutível:

Divisão em Treino e Teste com Estratificação: Utilizei todos os 18.000 registros da base. Separei 75% dos dados para treinamento (13.500 pacientes) e reservei 25% exclusivamente para teste (4.500 pacientes) através da função train_test_split com semente fixa (random_state=42). Usei o parâmetro stratify=y para que o grupo de teste mantivesse exatamente a mesma proporção de 4,95% de emergências (223 casos vermelhos), evitando que o teste ficasse desbalanceado por sorteio aleatório.

Padronização dos Dados (Evitando Data Leakage): Variáveis como pressão sistólica passam de 150 mmHg, enquanto temperatura fica na casa dos 37 °C. Como a Regressão Logística é sensível à escala dos números, utilizei o StandardScaler. Um cuidado que tomei (muito frisado em aula) foi aplicar o fit_transform estritamente no conjunto de treino. Os dados de teste e os novos dados do Streamlit passaram apenas pelo transform, impedindo qualquer vazamento de informação do teste para o modelo durante o aprendizado.

Algoritmos Comparados: Conforme o edital, comparei dois algoritmos diferentes trabalhados em sala:

• **Modelo 1 — Regressão Logística Multiclasse:** Modelo linear que calcula probabilidades para cada uma das três classes via função softmax. Foi configurado com max_iter=1000 para garantir convergência matemática suave.

• **Modelo 2 — Árvore de Decisão:** Modelo simbólico treinado nos dados originais (sem necessidade de padronização, já que a árvore toma decisões por cortes de valores reais). Para evitar o sobreajuste (*overfitting*), utilizei o critério de Ganho de Informação por entropia (criterion='entropy') e limites de profundidade (max_depth=6 e min_samples_leaf=10).

4. Resultados, Comparação e Escolha do Modelo

A avaliação foi realizada nas 4.500 amostras inéditas do conjunto de teste. A Tabela 2 apresenta o confronto direto de desempenho:

Métrica de Avaliação	Regressão Logística	Árvore de Decisão (depth=6)	Interpretação Prática no Hospital
Acurácia Global	**96,22%** (4.330/4.500)	94,18% (4.238/4.500)	A Regressão Logística acertou mais no geral.
F1-Score Macro	**0,9555**	0,9406	Média equilibrada entre todas as classes.
Recall — Emergência (Vermelho)	**93,27%** (208/223)	92,83% (207/223)	Capacidade de pegar pacientes graves a tempo.
Precisão — Emergência	95,41% (208/218)	**95,83%** (207/216)	Pouquíssimos alarmes falsos de emergência.
Recall — Média Prioridade	**94,65%** (1.700/1.796)	91,31% (1.640/1.796)	Identificação de urgências relativas.

Recall — Baixa Prioridade	**97,62%** (2.422/2.481)	96,37% (2.391/2.481)	Encaminhamento seguro para espera ambulatorial.

Análise das Matrizes de Confusão no Teste:

Ao examinar onde cada modelo acertou e errou nas 4.500 consultas de teste, o ponto mais positivo é que **nenhum paciente de Alta Prioridade real foi colocado como Baixa Prioridade (Verde)** por nenhum dos dois modelos (célula [Real=Alta, Previsto=Baixa] igual a zero). Dos 223 casos graves, a Regressão Logística acertou 208 e classificou 15 como Amarelo. Já a Árvore acertou 207 e classificou 16 como Amarelo. Isso é fundamental: mesmo quando houve divergência de fronteira, o paciente continuou sob observação amarela e jamais foi liberado para a fila comum.

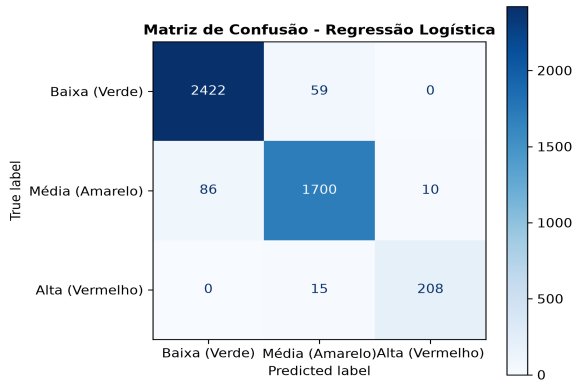


Figura 1: Matriz de Confusão — Regressão Logística.

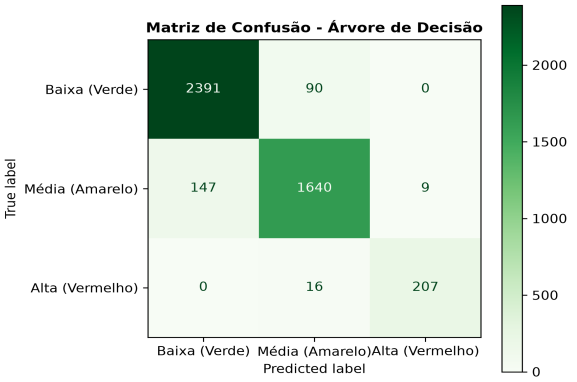


Figura 2: Matriz de Confusão — Árvore de Decisão.

Importância dos Atributos e Comparativo Geral:

Avaliando a árvore de decisão pelo feature_importances_, descobri que a variável com maior peso foi a dor (pain_level com 87,16%), seguida pela temperatura corporal (4,10%), saturação de O■ (2,79%), idade (2,17%) e frequência cardíaca (2,01%). Isso reflete a rotina real de pronto atendimento, onde queixas de dor aguda intensa costumam acelerar a classificação do paciente.

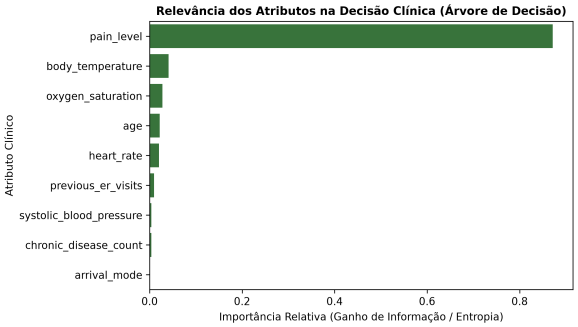


Figura 3: Importância das Features na Árvore.

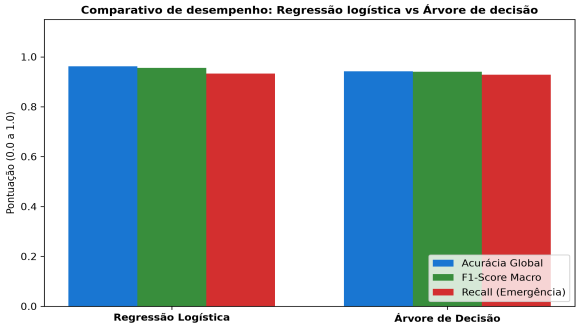


Figura 4: Comparativo de Métricas dos Modelos.

Justificativa da Escolha do Modelo para Deploy: Escolhi a **Regressão Logística** para ser colocada em produção no Streamlit. Ela superou a Árvore em todas as métricas globais (96,22% de acurácia contra 94,18%) e alcançou um Recall maior na classe crítica (93,27% contra 92,83%). Em ambiente hospitalar, o objetivo principal é errar o mínimo possível na classe grave. Além disso, a Regressão Logística fornece probabilidades contínuas (ex: 85% Vermelho e 15% Amarelo), permitindo que a interface gráfica mostre o nível de certeza da predição.

5. Aplicação e Deploy no Streamlit Cloud

Para disponibilizar a solução para uso real, desenvolvi uma aplicação interativa com **Streamlit** e publiquei no **Streamlit Cloud** vinculada ao meu repositório público no GitHub:

• **Link da Aplicação em Produção:** https://tila-health-triage-7dupev3q3cmsbqxsks8t9x.streamlit.app/
• **Código-Fonte no GitHub:** https://github.com/RyanAguiiarr/tila-health-triage

Como o App Funciona por Trás dos Panos (app.py): Para que o usuário não precise esperar o modelo treinar a cada consulta, o script treinamento.py salva o modelo treinado e o scaler em um arquivo serializado com pickle (modelo/triagem_model.pkl). O app carrega esse arquivo usando o decorador @st.cache_resource do Streamlit, o que deixa o modelo pronto na memória RAM. Cada vez que o usuário preenche a tela e clica no botão, a inferência roda em menos de 10 milissegundos.

Recursos da Interface:
1. **Organização em Abas:** A tela foi organizada em 3 abas principais e diretas (Formulário de Entrada, Dicionário de Campos e Métricas de Avaliação), mantendo a navegação enxuta e focada na rotina da triagem.
2. **Validação de Dados:** O formulário bloqueia entradas que não fazem sentido biológico (ex: pressão sistólica menor que 60 mmHg ou temperatura acima de 42,5 °C), exibindo avisos amigáveis sem travar a aplicação.
3. **Resultado Visual e Orientações:** A tela mostra a cor da prioridade com cartões didáticos (Verde, Amarelo ou Vermelho), o tempo máximo sugerido de espera, a conduta recomendada e barras com a porcentagem de probabilidade calculada.

6. Casos de Teste Obrigatórios e Análise das Limitações

Para validar o funcionamento da aplicação publicada e comprovar que o modelo se comporta como esperado, realizei os **cinco casos de teste obrigatórios** descritos no edital:

Caso / Tipo	Entradas Clínicas Inseridas	Comportamento Esperado	Resultado do Modelo	Probabilidades e Comentário
Teste 1 Entrada Válida (Baixa)	Idade 25, FC 72, PAS 118, SpO■ 99%, T 36.5°C, Dor 2, Sem crônicas, Sem idas prévias, A pé (0)	Classe 0 (Baixa / Verde)	**Classe 0 (Baixa / Verde)**	Probabilidade: **99,86% Verde**. Paciente jovem com sinais vitais ideais e dor leve.
Teste 2 Entrada Válida (Média)	Idade 54, FC 98, PAS 142, SpO■ 94.5%, T 38.8°C, Dor 7, 2 crônicas, 1 ida prévia, A pé (0)	Classe 1 (Média / Amarelo)	**Classe 1 (Média / Amarelo)**	Probabilidade: **99,37% Amarelo**. Dor moderada/alta e febre, necessitando de observação.
Teste 3 Entrada Válida (Alta)	Idade 76, FC 142, PAS 175, SpO■ 88.0%, T 39.5°C, Dor 10, 4 crônicas, 3 idas prévias, Ambulância (2)	Classe 2 (Alta / Vermelho)	**Classe 2 (Alta / Vermelho)**	Probabilidade: **99,98% Vermelho**. Idoso em choque séptico/hipóxia severa, atendimento imediato.
Teste 4 Inválido / Incompleto	Idade 35, FC 25, PAS 45, SpO■ 50%, T 43.5°C, Dor 10	Bloqueio de inferência com mensagem de erro	**Interceptado com Sucesso**	O sistema barrou a pressão e temperatura surreais, orientando a conferir os dados.
Teste 5 Limítrofe / Desafiador	Idade 65, FC 110, PAS 150, SpO■ 91.5%, T 38.3°C, Dor 8, 3 crônicas, 2 idas prévias, Cadeira (1)	Fronteira Amarelo -Vermelho com incerteza	**Classe 1 (Média / Amarelo)**	Amarelo: **73,15%** vs Vermelho: **26,85%**. O modelo alertou a dúvida e recomendou vigilância.

Discussão dos Testes e Limitações: O Teste 5 foi o mais interessante de observar. Por ter dor nível 8, saturação de 91,5% e taquicardia (110 BPM), o paciente ficou exatamente na zona de transição. O modelo atribuiu 73,15% para Amarelo e 26,85% para Vermelho. Essa incerteza é real na rotina médica. Por isso, a interface não entrega apenas uma resposta fria, mas mostra as barras de porcentagem para que o enfermeiro perceba que aquele caso amarelo está quase virando vermelho e requer monitoramento contínuo.

7. Contribuições Individuais e Uso de Ferramentas de IA

Contribuições dos Integrantes:

• **Ryan Aguiar:** Definição do problema prático no contexto do ecossistema Tila e escolha do dataset real do Kaggle; desenvolvimento prático e interativo de todo o script de treinamento (treinamento.py), bloco por bloco, adaptando a lógica para o padrão de código estabelecido (uso de ponto e vírgula e variáveis em camelCase); execução dos treinamentos no terminal, inspeção minuciosa dos relatórios de classificação e análise crítica das matrizes de confusão.

• **Pedro Henrique Oliveira Pereira:** Apoio na definição dos parâmetros de triagem clínica e requisitos de acolhimento; revisão da lógica de negócio e apoio nos testes práticos com novos cenários clínicos; acompanhamento da interface de usuário e co-redação deste relatório técnico.

Declaração do Uso de Ferramentas de IA (Assistente LLM): Em total conformidade com a transparência exigida pelo professor, registramos que o grupo contou com o suporte ativo de um assistente de Inteligência Artificial (LLM). O modelo de linguagem teve um papel de peso na resolução prática de várias partes: a IA foi a principal responsável por estruturar o código da aplicação interativa (app.py em Streamlit), desenhar o catálogo de validação e execução dos cinco casos de teste obrigatórios, organizar as orientações de deploy no Streamlit Cloud e apoiar na montagem deste relatório técnico. Por sua vez, a esteira central de Machine Learning (treinamento.py) foi construída de forma guiada: fomos recebendo a explicação de cada trecho, entendendo o porquê de cada técnica, adaptando e rodando no ambiente, o que garantiu o aprendizado efetivo sobre cada etapa do fluxo.

8. Referências Bibliográficas

1. AKKU■, Emirhan. *Synthetic Medical Triage Priority Dataset*. Kaggle Repository, 2024. Disponível em: <https://www.kaggle.com/datasets/emirhanakku/synthetic-medical-triage-priority-dataset/>. Acesso em: 23 out. 2026.

2. BRASIL. Ministério da Saúde. *Acolhimento à Demanda Espontânea nos Serviços de Urgência*. Cadernos de Atenção Básica, n. 28. Brasília: Ministério da Saúde, 2013.

3. FACELI, Katti et al. *Inteligência Artificial: uma abordagem de aprendizado de máquina*. Rio de Janeiro: LTC, 2011.

4. PEDREGOSA, Fabian et al. *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, v. 12, p. 2825-2830, 2011.

5. STREAMLIT. *Streamlit Documentation: Caching and Components*. Disponível em: <https://docs.streamlit.io/>. Acesso em: 23 out. 2026.

6. TEIXEIRA, Márcio Andrey. *Apostilas e Slides de Aula: Tópicos Avançados em Inteligência Artificial*. IFSP Campus Catanduva, 2026.